

Python Dictionary (Module-8)

Dictionaries are used to store data values in key: value pairs. A dictionary is a collection which is ordered*, changeable and do not allow duplicate keys. As of Python version 3.7, dictionaries are ordered. In Python 3.6 and earlier, dictionaries are unordered.

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
print(thisdict)
```

Output

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
```

Dictionary Items - Data Types

The values in dictionary items can be of any data type:

```
thisdict = {  
    "brand": "Ford",  
    "electric": False,  
    "year": 1964,  
    "colors": ["red", "white", "blue"]  
}
```

The dict() Constructor

It is also possible to use the dict() constructor to make a dictionary.

```
thisdict = dict(name = "John", age = 36, country = "Norway")  
print(thisdict)
```

Output

```
{'name': 'John', 'age': 36, 'country': 'Norway'}
```

Accessing Items

You can access the items of a dictionary by referring to its key name, inside square brackets:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
x = thisdict["model"]  
x = thisdict.get("model")
```

Output

```
Mustang
```

Adding Items

Adding an item to the dictionary is done by using a new index key and assigning a value to it:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict["color"] = "red"  
print(thisdict)
```

Output

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964, 'color': 'red'}
```

Change Values

You can change the value of a specific item by referring to its key name:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
  
thisdict["year"] = 2018  
print(thisdict)
```

Output

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 2018}
```

Nested Dictionaries

A dictionary can contain dictionaries, this is called nested dictionaries.

```
myfamily = {  
    "child1" : {  
        "name" : "Emil",  
        "year" : 2004  
    },  
    "child2" : {  
        "name" : "Tobias",  
        "year" : 2007  
    },  
    "child3" : {  
        "name" : "Linus",  
        "year" : 2011  
    }  
}  
  
print(myfamily)
```

Output

```
{'child1': {'name': 'Emil', 'year': 2004}, 'child2': {'name': 'Tobias', 'year': 2007}, 'child3': {'name': 'Linus', 'year': 2011}}
```

Checking the existence of a key in dictionary

We can use in operator to check the existence of a key in the dictionary.

```
d = {"a": 1, "b": 2, "c": 3}
if "a" in d:
    print("Key exists")
else:
    print("Key does not exist")
```

Output

Key exists

Loop through a Dictionary

You can loop through a dictionary by using a **‘for’** loop. When looping through a dictionary, the return value are the *keys* of the dictionary, but there are methods to return the *values* as well.

```
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
for x in thisdict:
    print(x)
```

Output

brand
model
year

Print all *values* in the dictionary, one by one:

```
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
for x in thisdict:
    print(thisdict[x])
```

Output

Ford
Mustang
1964

You can also use the values() method to return values of a dictionary:

```
for x in thisdict.values():  
    print(x)
```

Output

```
Ford  
Mustang  
1964
```

You can use the keys() method to return the keys of a dictionary:

```
for x in thisdict.keys():  
    print(x)
```

Output

```
brand  
model  
year
```

Loop through both keys and values, by using the items() method:

```
for x, y in thisdict.items():  
    print(x, y)
```

Output

```
brand Ford  
model Mustang  
year 1964
```

Python Dictionary Methods

clear()

The clear() method removes all the elements from a dictionary.

Syntax

```
dictionary.clear()
```

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
car.clear()  
print(car)
```

Output

```
{}
```

copy()

The copy() method returns a copy of the specified dictionary.

Syntax

dictionary.copy()

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

```
x = car.copy()  
print(x)
```

Output

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
```

fromkeys()

The fromkeys() method returns a dictionary with the specified keys and the specified value.

Syntax

dict.fromkeys(keys, [value])

The default value of parameter 'value' is None.

```
x = ('key1', 'key2', 'key3')  
y=1  
thisdict = dict.fromkeys(x,y)  
print(thisdict)
```

Output

```
{'key1': 1, 'key2': 1, 'key3': 1}
```

get()

The get() method returns the value of the item with the specified key.

Syntax

dictionary.get(keyname, [value])

Value - A value to return if the specified key does not exist. Default value None.

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

```
x = car.get("model")  
print(x)
```

Output:

Mustang

items()

The items() method returns a view object. The view object contains the key-value pairs of the dictionary, as tuples in a list.

Syntax

dictionary.items()

keys()

The keys() method returns a view object. The view object contains the keys of the dictionary, as a list.

values()

The values() method returns a view object. The view object contains the values of the dictionary, as a list.

Syntax

dictionary.values()

pop()

The pop() method removes the specified item from the dictionary.

The value of the removed item is the return value of the pop() method, see example below.

Syntax

dictionary.pop(keyname, [defaultvalue])

defaultvalue - A value to return if the specified key do not exist.

If this parameter is not specified, and the no item with the specified key is found, an error is raised

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
x = car.pop("model")  
print(x)
```

Output

Mustang

popitem()

The popitem() method removes the item that was last inserted into the dictionary. In versions before 3.7, the popitem() method removes a random item. The removed item is the return value of the popitem() method, as a tuple, see example below.

Syntax

dictionary.popitem()

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

```
car.popitem()
print(car)
```

Output

```
{'brand': 'Ford', 'model': 'Mustang'}
```

setdefault()

The setdefault() method returns the value of the item with the specified key.

If the key does not exist, insert the key, with the specified value, see example below

Syntax

dictionary.setdefault(keyname, [value])

value - If the key exist, this parameter has no effect. If the key does not exist, this value becomes the key's value Default value None

```
car = {
    "brand": "Ford",
    "year": 1964
}
car.setdefault("model", "Bronco")
print(car)
```

Output:

```
{'brand': 'Ford', 'year': 1964, 'model': 'Bronco'}
```

update()

The update() method inserts the specified items to the dictionary. The specified items can be a dictionary, or an iterable object with key value pairs.

Syntax

dictionary.update(iterable)

```
car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
car.update({"color": "White"})
print(car)
```

Output:

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964, 'color': 'White'}
```

Finding maximum, minimum values from a dictionary and finding sum of all the values in the dictionary:

```
d = {"n1": 20, "n2": 10, "n3": 80, "n4": 70, "n5": 60}
print(max(d.values()))
print(min(d.values()))
print(sum(d.values()))
```

Output:

```
80
10
240
```